

11. Se dau 2 siruri de octeti A si B. Sa se construiasca sirul R care sa contina doar elementele impare si pozitive din cele 2 siruri.

Exemplu:

A: 2, 1, 3, -3

B: 4, 5, -5, 7

R: 1, 3, 5, 7

assume cs:code, ds:data

data segment

a db 2, 1, 3, -3

len1 equ \$ - a

b db 4, 5, -5, 7

len2 equ \$ - b

r db 20 dup(?)

len3 dw ?

data ends

code segment

start:

mov ax, data

mov ds, ax

mov si, 0 ; index in a

mov di, 0 ; index in r

processA:

cmp si, len1 ; si >= len1 ?

jge finalA ; yes ? stop processing A : continue

mov al, a[si] ; al = elem of a[]

cmp al, 0 ; pos

jle skipA ; al <= 0 skip

test al, 1 ; odd

je skipA ; even skip

mov r[di], al

inc di

skipA:

inc si

jmp processA ; step to repeat

finalA:

mov si, 0 ; now process B

processB:

cmp si, len2 ; si >= len2 ?

jge finalB ; yes ? stop processing B : continue

mov al, b[si] ; al = elem of b[]

cmp al, 0 ; pos

je skipB

test al, 1 ; odd
je skipB

mov r[di], al
inc di

skipB:
inc si
jmp processB

finalB:
mov len3, di

mov ax, 4C00h
int 21h

code ends
end start

$$\begin{aligned} ((20 * 12) + 2005) \% 15 + 1 &= 11 \quad (PB1) \\ ((2005 * 12) + 20) \% 15 + 1 &= 6 \quad (PB2) \end{aligned}$$

11. Se dau 2 siruri de cifre A si B. Se ce construieste
sirul R care sa contina toate elementele impare
si pozitive din cele 2 siruri.

Ex: A: 2, 1, 3, -3 1, 3, 5, 7
B: 4, 5, -5, 7
R: 1, 3, 5, 7

6. Se da un sir S de dublu-cuvinte.
Se ce obtine sirul D format din cifre impare
si cuvintele impare din elementele sirului de
dublu-cuvinte, care sunt multipli de 7.

Ex: s DB 1234567h, 1A2B3C15h, BA33412h
d DB 07h, 15h

11
separate app: $a[] = 2, 1, 3, -3$, len1=4
 $b[] = 4, 5, -5, 7$, len2=4
 $r[]$: , len3=0
while(len1--){
if(a[len1]>0 && a[len1]%2==1)
r[len3++] = a[len1];
}
while(len2--){
if(b[len2]>0 && b[len2]%2==1)
r[len3++] = b[len2];
}

$\Rightarrow a$ de 2, 1, 3, -3 $\Rightarrow$ len1 egu $\$ - a$
 b de 4, 5, -5, 7 $\Rightarrow$ len2 egu $\$ - b$

$\rightarrow$ in ex punem nr de elemente :

• Cazuri : • toate nr din a si din b sunt poz & comp
 $\Rightarrow$ nr elemente din while/for loop
 este len1 + len2

• ccaun si nr poz si nr neg dar
 $\Rightarrow$ nr elemente este tot len1 + len2
 deoarece este suficient pentru

~~Dei rezervam pt r [] o dimensiune = len1 + len2~~

Dei rezervam spatiu pt r [] lor nr de elemente
 este len1 + len2.

$\rightarrow r$ de 20 dup (?)
 len3 din ?

$si \rightarrow r$ returna ca index 0 in a []
 $di \rightarrow$ index 0 in r []

Procedura A : ; verificam ca si sa fie $< len1$

comp si, len1

je finalA $\rightarrow$ iesim din loop

mov al, a[si]

comp al, 0 ; pozitie

je skipA $\rightarrow$ r exclusiv

test al, 1 ; imparte
 je skipA

~~mov r[di], al~~

mov r[di], al

inc di ; incrementam indexul in r

skip A :

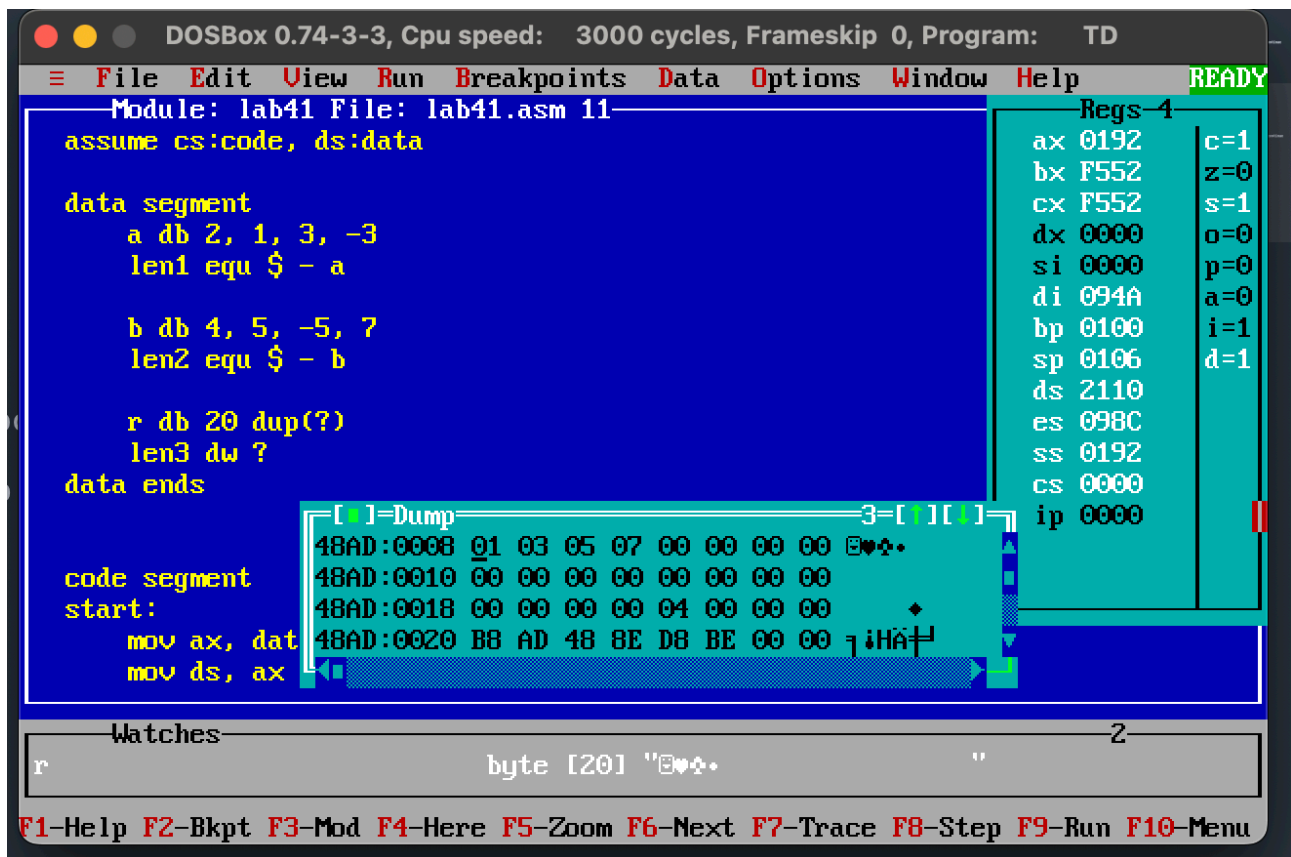
inc si ; incrementam indexul in a

jmp procesA ; revenim la loop-ul initial

final A :

mov si, 0 ; ne mutam in b

$\rightarrow$ pentru b facem exact aceasi pasi si la final
 punem in len3, di



6. Se da un sir S de dublucuvinte.

Sa se obtina sirul D format din octetii inferiori ai cuvintelor inferioare din elementele sirului de dublucuvinte, care sunt multiplii de 7.

Exemplu:

s DD 12345607h, 1A2B3C15h, 13A33412h

d DB 07h, 15h

```
assume cs:code, ds:data
```

```
data segment
```

```
    s dd 12345607h, 1A2B3C15h, 13A33412h
```

```
    len_s equ ($ - s) / 4
```

```
    d db 10 dup(?)
```

```
    len_d dw ?
```

```
data ends
```

```
code segment
```

```
start:
```

```
    mov ax, data
```

```
    mov ds, ax
```

```
    mov si, 0    ; si = index 0 in s
```

```
    mov di, 0    ; di = index 0 in d
```


mov cx, len_s ; cx = iterations

process_element:

mov al, byte ptr s[si] ; al = 01

mov ah, 0

mov bl, 7 ; div with 7

div bl

cmp ah, 0

jne skip_copy

mov al, byte ptr s[si] ; reload al to check the division by 7

mov d[di], al ; save result

inc di

skip_copy:

add si, 4

loop process_element ; go back if cx != 0

final:

mov len_d, di

mov ax, 4C00h

int 21h

code ends

end start

6. ~~s~~ ~~d~~ 1234567h, 1A2B3C15h, 13A33412h
d ~~d~~ → multiplu de 7
↓
len-s
↓
len-d

⇒ poziționăm s pe indexul 0 în s[]
poziționăm d pe indexul 0 în d[]

• în cx avem nr de iterații ⇒ $cx = len - s$

• în al avem restul cel mai puțin semnificativ
⇒ $al = \text{restul de la } (s \text{ la } len - s)$
verificăm dacă e multiplu de 7 (AX/BL)

• donc restant de l'opération $= 0 \Leftrightarrow AH=0$
multiple de 7

$$\left. \begin{array}{l} AX/BX \rightarrow AL = \text{rest} \\ AH = \text{rest} \end{array} \right\} AH=0$$

$\rightarrow$ premier val en d + l'incrément de index

The screenshot shows the DOSBox 0.74-3-3 interface. The top menu bar includes File, Edit, View, Run, Breakpoints, Data, Options, Window, and Help. The status bar at the bottom shows F1-Help, F2-Bkpt, F3-Mod, F4-Here, F5-Zoom, F6-Next, F7-Trace, F8-Step, F9-Run, and F10-Menu.

The main window displays assembly code for a module named 'lab42.asm'. The code includes a data segment with a double word array 's' and a code segment with a 'start' label. The assembly code is as follows:

```

Module: lab42 File: lab42.asm 11
assume cs:code, ds:data

data segment
    s dd 12345607h, 1A2B3C15h, 13A33412h
    len_s equ ($ - s) / 4

    d db 10 dup(?)
    len_d dw ?
data ends

code segment
start:
    mov ax,
    mov ds,

    mov si, 0      ; si = index 0 in s
    mov di, 0      ; di = index 0 in d
  
```

A memory dump window is open, showing the contents of memory starting at address 48AD:000C. The dump shows the following hex values and their corresponding ASCII characters:

Address	Hex	ASCII
48AD:000C	07 15 00 00 00 00 00 00	07 15
48AD:0014	00 00 02 00 00 00 00 00	02
48AD:001C	00 00 00 00 BB AD 48 BE	BB AD 48 BE
48AD:0024	D8 BE 00 00 BF 00 00 B9	D8 BE 00 00 BF 00 00 B9

The right-hand pane shows the register values:

Register	Value
ax	0192
bx	F552
cx	F552
dx	0000
si	0000
di	094A
bp	0100
sp	0106
ds	2110
es	098C
ss	0192
cs	0000
ip	0000

The bottom status bar shows the current address 0000:0000 and the instruction pointer 0000.